

Unit-III

Trees: Binary tree, Terminology, Representation, Traversals, Applications – Binary search tree – AVL tree. B Trees: B Tree indexing, operations on a B Tree, Lower and upper bounds of a B Tree - B + Tree indexing – Trie Tree Indexing.

2 MARKS

1. Compare tree and Binary Tree (Nov-2016)

TREE	BINARY TREE
1. In a tree a node can have many children node.	1. In binary tree a node can have at most two nodes
2. A general tree cannot be empty.	2. A binary tree can be empty.

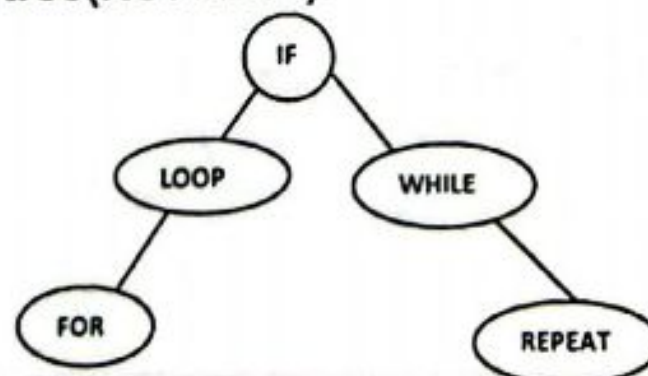
2. What is AVL Tree? (Nov-2016)

An AVL tree (Adelson-Velskii and Landis' tree, named after the inventors) is a self-balancing binary search tree.

3. What is the maximum number of nodes in a binary tree of depth d? (Nov-2014).

The maximum number of nodes at level 'l' of a binary tree is 2^l . Here level is the number of nodes on the path from the root to the node (including root and node).

4. Consider the reserved words (if,while,loop,repeat,for). Draw any two possible binary search tree(Nov-2014).



5. Define AVL tree. What do you mean by the balance factor of AVL tree (May-2018).

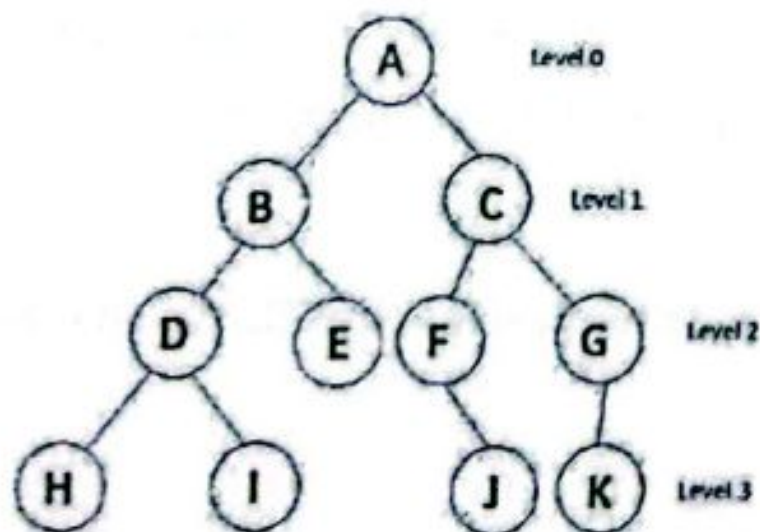
An AVL tree (Adelson-Velskii and Landis' tree, named after the inventors) is a self-balancing binary search tree. The balancing factor of an AVL tree is 0, 1 and -1. It is calculated as the difference between Left subtree and Right Subtree.

6. Compare B tree and B+ Tree (May-2018)

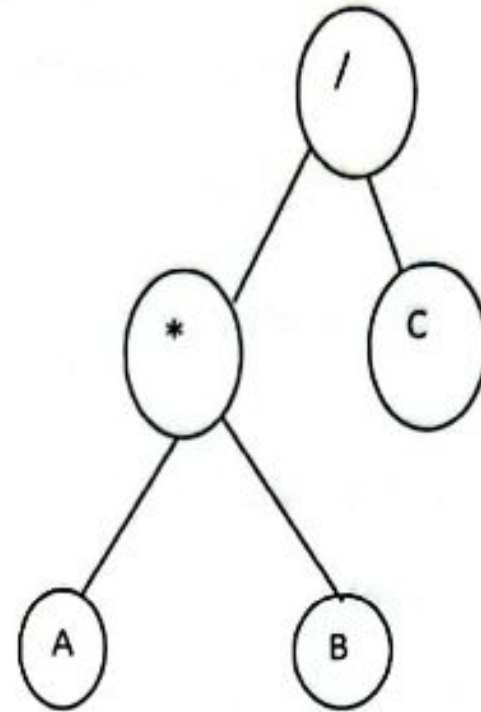
B TREE	B+ TREE
1. B-trees are similar to B+ trees but it allows search key values to appear only once, eliminates redundant storage of search keys. 2. It is possible sometimes to find search-key value before reaching the leaf node.	1. B+ tree is the form of a balanced tree in which every path from the root of the tree to a leaf of the tree is the same length. 2. B+ trees are good for searches but cause some overhead issues in wasted space.

7. Define Tree. Give an Example (April-2017)

Tree is a non-linear data structure. There is no linear relation between the data items. There is a special node design at the root node. The remaining nodes are partitioned into subtrees of a tree.



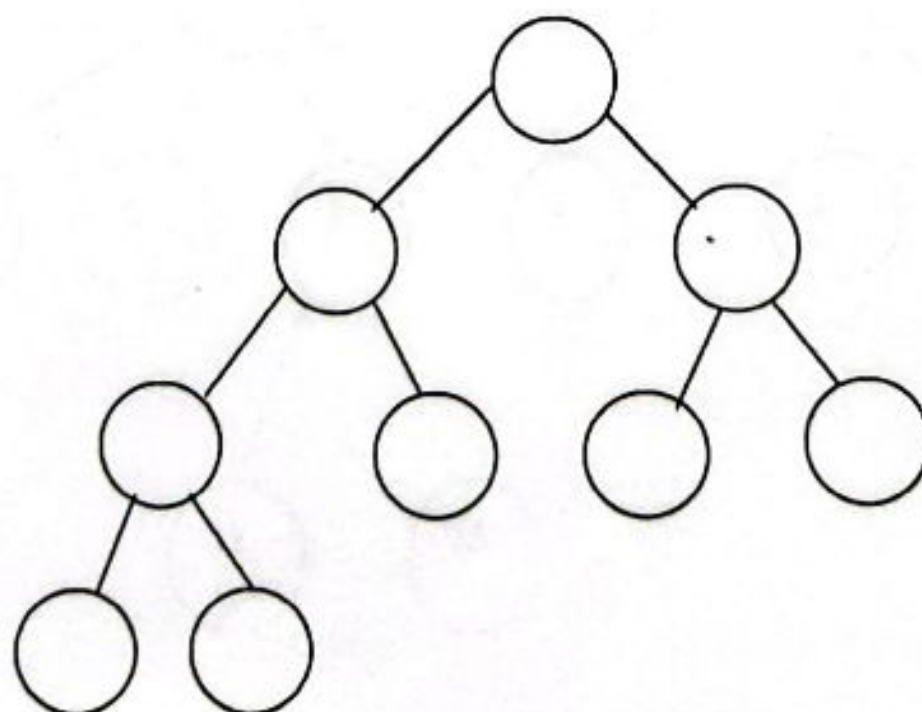
8. Draw the expression tree for the postfix expression AB^*C^+ using Stack (April-2017).



9. Differentiate between AVL Tree and B trees. (Nov-2019).

AVL TREE	B TREE
1. It is a Height balancing tree with balancing factor 0, 1, -1.	1. B-tree is a balancing tree but not a binary tree.
2. It is possible sometimes to find search-key value before reaching the leaf node.	2. They are used for storage backed tree.

10. Draw a Binary tree with 9 nodes (Nov 2017).



11. List out any 2 advantage of Threaded binary tree (Nov-2017).

(a) Threaded binary tree makes the In order traversal faster and do it without stack and without recursion.

(b) We can reduce memory space used to store a tree and also time.

12. What is complete Binary tree (Nov-2015).

A complete binary tree is a tree in which every non-leaf node has exactly two children not necessarily to be on the same level.

13. Write the properties of B trees (Nov-2015).

1. All leaves are at the same level.
2. All nodes may contain at most $t-1$ keys
3. Number of children of a node is equal to the number of keys in it plus 1.

11 Marks

1. How can you represent the set as a tree? What are the operations that we can perform on these sets? How these operations are implemented? (Nov- 2014).

In the tree representation of a set, a tree is used to represent one set, and each element in the set has same root. The trees are not necessarily binary trees. For example, let us consider three sets:

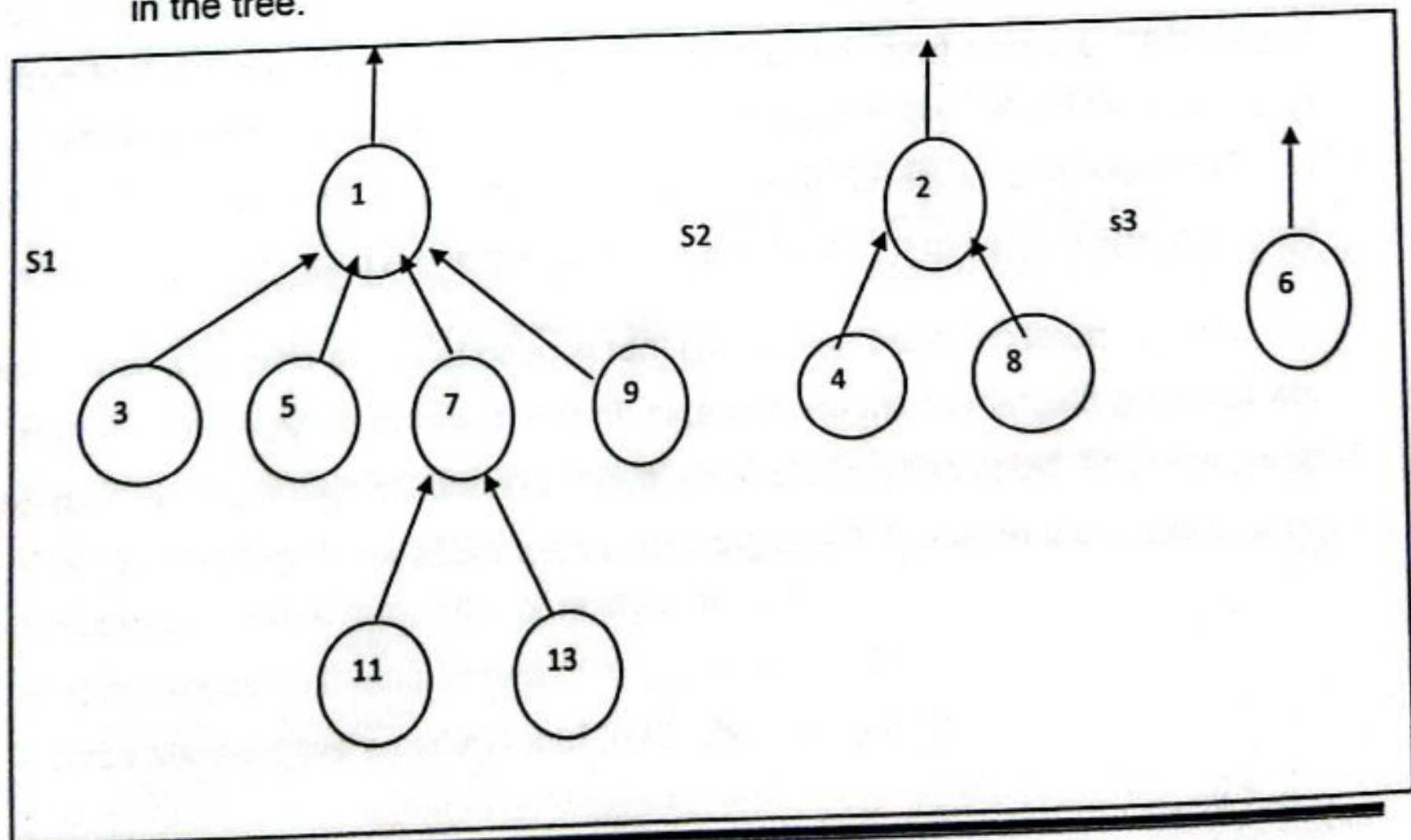
$$S1 = \{1, 3, 5, 7, 9, 11, 13\}$$

$$S2 = \{2, 4, 8\}$$

$$S3 = \{6\}$$

Tree representation of the above sets may appear as shown :

In the tree representation of sets, each element in a set has a pointer to its parent. For convenience, we draw the roots parent pointer vertically. The name of the set is given by the node at the root, in general. An array is sufficient to store the tree form as a set. We will assume that the elements are numbered as integers and the indices of the array indicate the element. The array content of the i th index is the parent of the i th element in the tree.



$s1 = \{1, 3, 5, 7, 9, 11, 13\}$

$s2 = \{2, 4, 8\}$

$s3 = \{6\}$

-	-	-	-	-	0	-	-	-	-	-	-	-	-	-	-
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

(a). Representation of set3 $s3 = \{6\}$

1	-	3	-	5	-	7	-	9	-	11	-	13	-	-	-
---	---	---	---	---	---	---	---	---	---	----	---	----	---	---	---

(b). Representation of set1 $s1 = \{1, 3, 5, 7, 9, 11, 13\}$

The other representation of set in tree are:

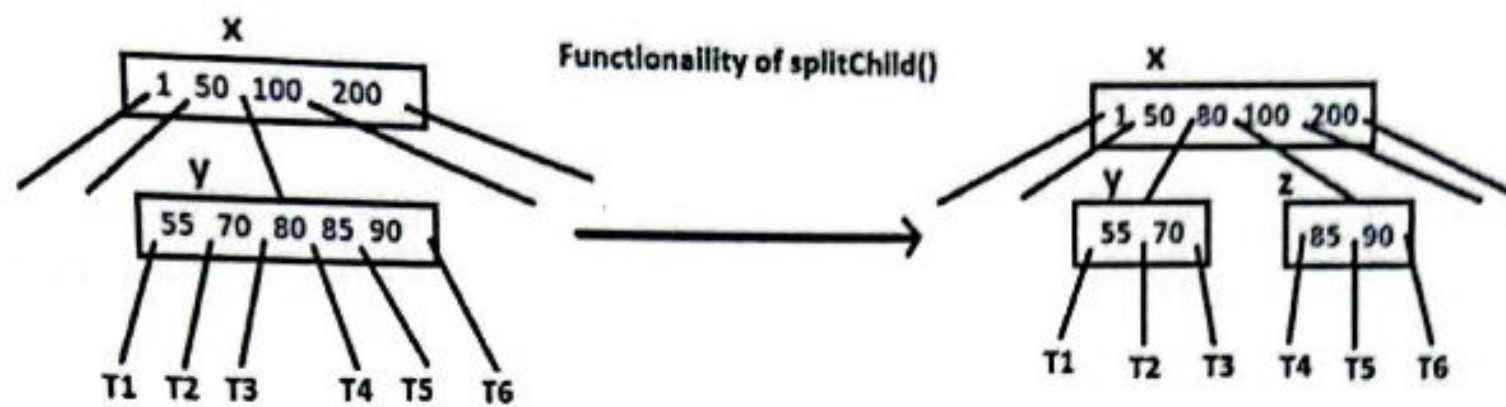
1. Union: It is the union of two sets $s1, s2$.
2. Intersection: It is searching a list for each element in the other list.
3. Difference: Suppose two sets S_i and S_j in the form of linked lists are given and we are to find $s_i - s_j$, the difference of s_j from s_i .
4. Inclusion: Adding the element x if it does not exist already.
5. Exclusion: We eliminate an element x if it is there, if the element occurs multiple times.
6. Equality: Equality of two sets s_i and s_j tests if $s_i == s_j$.

2. Describe the Insertion algorithm for B-Trees (Nov-2014)

A new key is always inserted at the leaf node. Let the key to be inserted be k . Like BST, we start from the root and traverse down till we reach a leaf node. Once we reach a leaf node, we insert the key in that leaf node. Unlike BSTs, we have a predefined range on the number of keys that a node can contain. So before inserting a key to the node, we make sure that the node has extra space.

We use an operation called `splitChild()` that is used to split a child of a node. See the following diagram to understand split. In the following diagram, child y of x is being split into two nodes y and z . Note that the `splitChild` operation moves a key up and this is the reason B-Trees grow up, unlike BSTs which grow down.

BTreeSplit



Insertion algorithm.

- 1) Initialize x as root.
- 2) While x is not leaf, do following
 - a) Find the child of x that is going to be traversed next. Let the child be y.
 - b) If y is not full, change x to point to y.
 - c) If y is full, split it and change x to point to one of the two parts of y. If k is smaller than mid key in y, then set x as the first part of y. Else second part of y. When we split y, we move a key from y to its parent x.
- 3) The loop in step 2 stops when x is leaf. x must have space for 1 extra key as we have been splitting all nodes in advance. So simply insert k to x.

3. Illustrate the deletion operation on B+ tree and explain (Nov-2014)

Deleting an element on a B+ tree consists of three main events: searching the node where the key to be deleted exists, deleting the key and balancing the tree if required. Underflow is a situation when there is less number of keys in a node than the minimum number of keys it should hold.

Deletion Operation

Before going through the steps below, one must know these facts about a B+ tree of degree m.

A node can have a maximum of m children. (i.e. 3)

A node can contain a maximum of m - 1 keys. (i.e. 2)

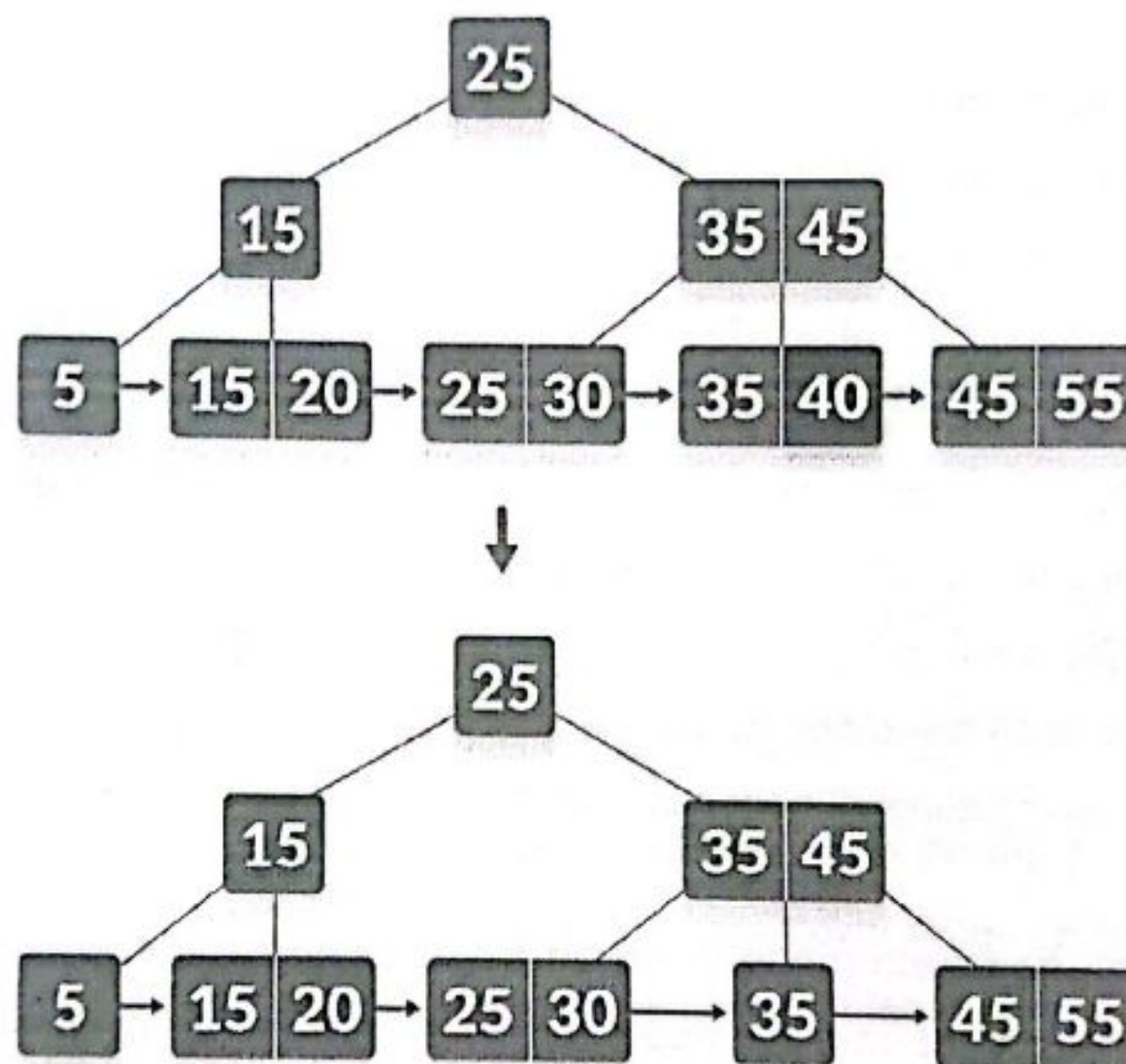
A node should have a minimum of $\lceil m/2 \rceil$ children. (i.e. 2)

A node (except root node) should contain a minimum of $\lceil m/2 \rceil - 1$ keys. (i.e. 1)

While deleting a key, we have to take care of the keys present in the internal nodes (i.e. indexes) as well because the values are redundant in a B+ tree. Search the key to be deleted then follow the following steps.

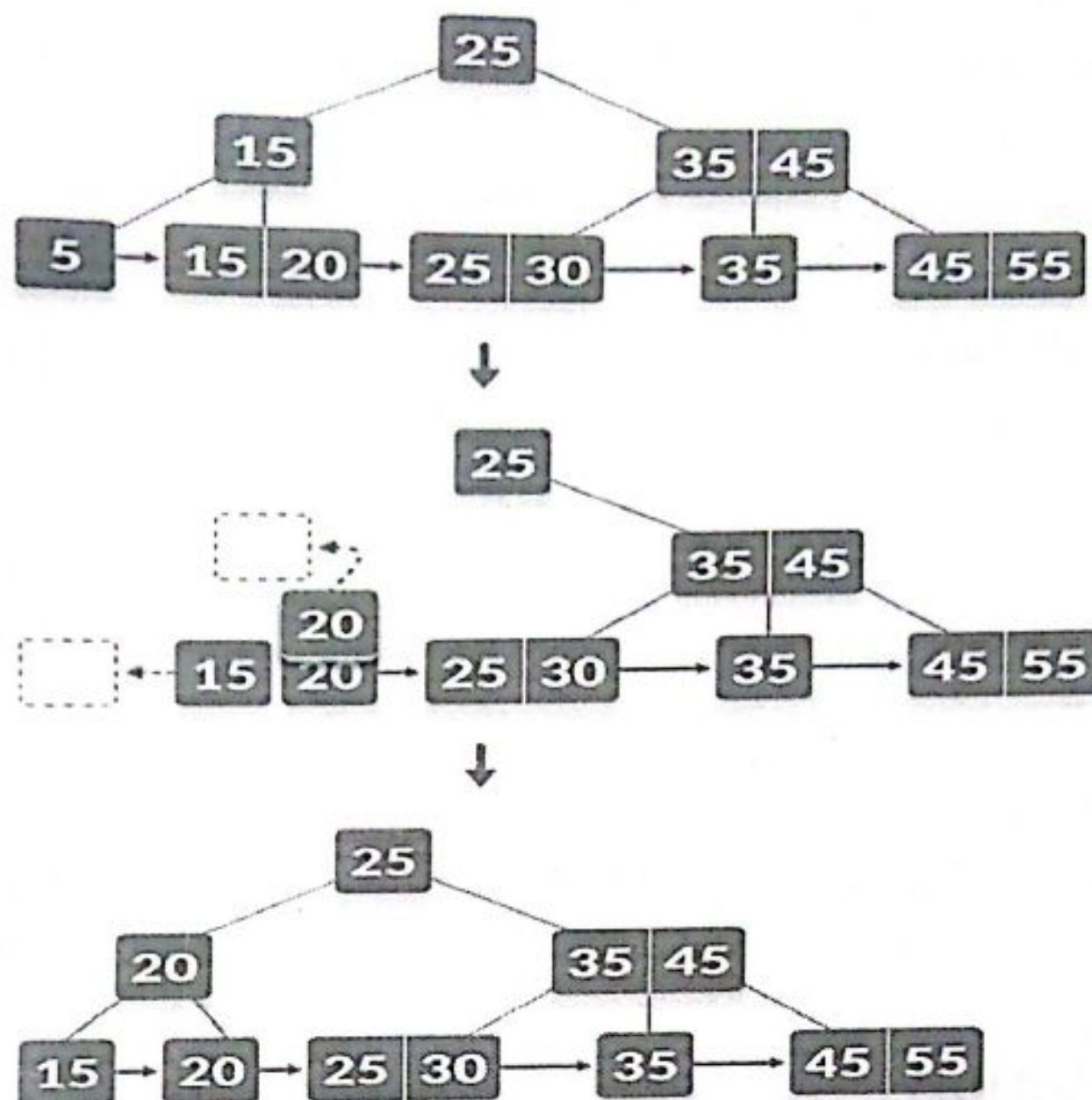
Case I

The key to be deleted is present only at the leaf node not in the indexes (or internal nodes). There are two cases for it:



There is more than the minimum number of keys in the node. Simply delete the key.

2. There is an exact minimum number of keys in the node. Delete the key and borrow a key from the immediate sibling. Add the median key of the sibling node to the parent.



4. How binary trees can be represented? Explain the various binary tree traversal algorithms with example (Dec-2016, Nov-2019)

Unlike linear data structures (Array, Linked List, Queues, Stacks, etc) which have only one logical way to traverse them, trees can be traversed in different ways. Following are the generally used ways for traversing trees.

Algorithm Inorder(tree)

1. Traverse the left subtree, i.e., call Inorder(left-subtree)
2. Visit the root.
3. Traverse the right subtree, i.e., call Inorder(right subtree)

Preorder Traversal (Practice):

Algorithm Preorder(tree)

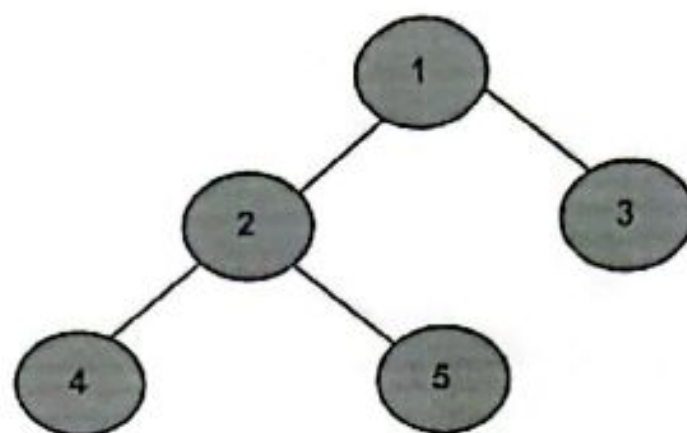
1. Visit the root.
2. Traverse the left subtree, i.e., call Preorder(left-subtree)

3. Traverse the right subtree, i.e., call Preorder(right-subtree)

Postorder Traversal (Practice):

Algorithm Postorder(tree)

1. Traverse the left subtree, i.e., call Preorder(left-subtree)
2. Traverse the right subtree, i.e., call Preorder(right-subtree)
3. Visit the root.



(a) Inorder (Left, Root, Right) : 4 2 5 1 3

(b) Preorder (Root, Left, Right) : 1 2 4 5 3

(c) Postorder (Left, Right, Root) : 4 5 2 3 1

5. What is B tree? Discuss in detail.(Dec-2016).

B-Tree is a self-balancing search tree. In most of the other self-balancing search trees (like AVL and Red-Black Trees), it is assumed that everything is in main memory. To understand the use of B-Trees, we must think of the huge amount of data that cannot fit in main memory. When the number of keys is high, the data is read from disk in the form of blocks. Disk access time is very high compared to the main memory access time. The main idea of using B-Trees is to reduce the number of disk accesses. Most of the tree operations (search, insert, delete, max, min, etc) require $O(h)$ disk accesses where h is the height of the tree. B-tree is a fat tree. The height of B-Trees is kept low by putting maximum possible keys in a B-Tree node. Generally, the B-Tree node size is kept equal to the disk block size. Since the height of the B-tree is low so total disk accesses for most of the operations are reduced significantly compared to balanced Binary Search Trees like AVL Tree, Red-Black Tree, etc.

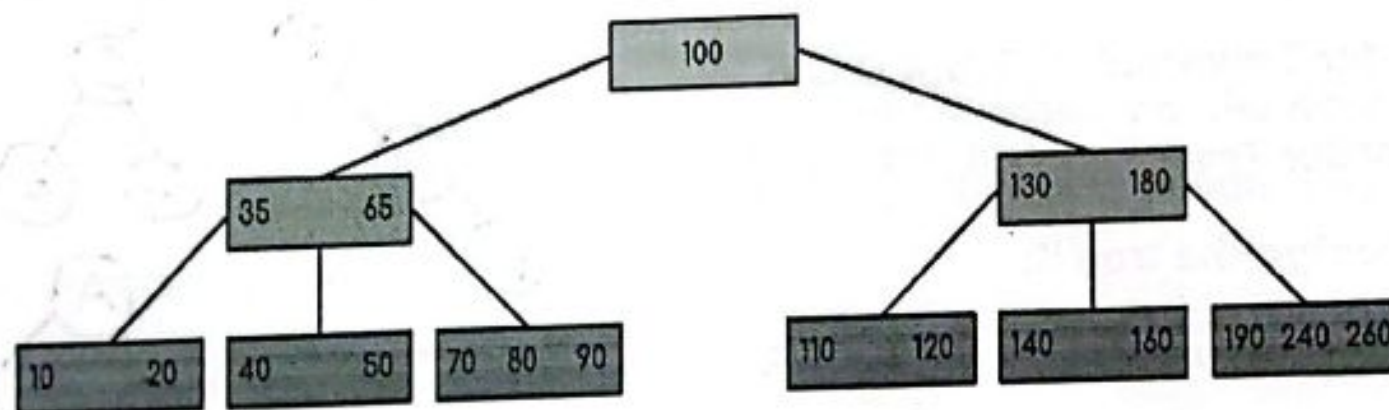
Properties of B-Tree:

1. All leaves are at the same level.
2. A B-Tree is defined by the term minimum degree 't'. The value of t depends upon disk block size.
3. Every node except root must contain at least $\lceil (t-1)/2 \rceil$ keys. The root may contain minimum 1 key.
4. All nodes (including root) may contain at most $t - 1$ keys.
5. Number of children of a node is equal to the number of keys in it plus 1.
6. All keys of a node are sorted in increasing order. The child between two keys k_1 and k_2 contains all keys in the range from k_1 and k_2 .

B-Tree grows and shrinks from the root which is unlike Binary Search Tree. Binary Search Trees grow downward and also shrink from downward.

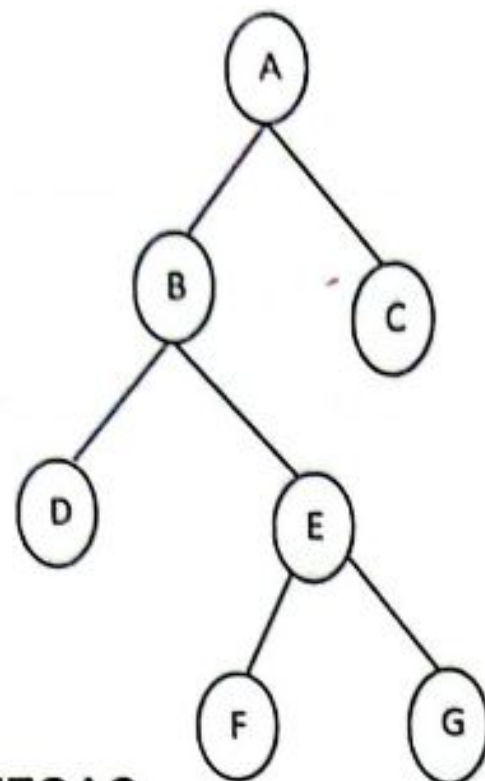
Like other balanced Binary Search Trees, time complexity to search, insert and delete is $O(\log n)$.

Following is an example of B-Tree of minimum order 5. Note that in practical B-Trees, the value of the minimum order is much more than 5.



We can see in the above diagram that all the leaf nodes are at the same level and all non-leaf have no empty sub-tree and have keys one less than the number of their children.

6. Solve the following: Traverse the tree Inorder, Preorder and Postorder. (May-2018)



Inorder traversal: DBFEGAC

Preorder traversal: ABDEFGC

Postorder traversal: DFGEBAC

(B) Design the tree if :

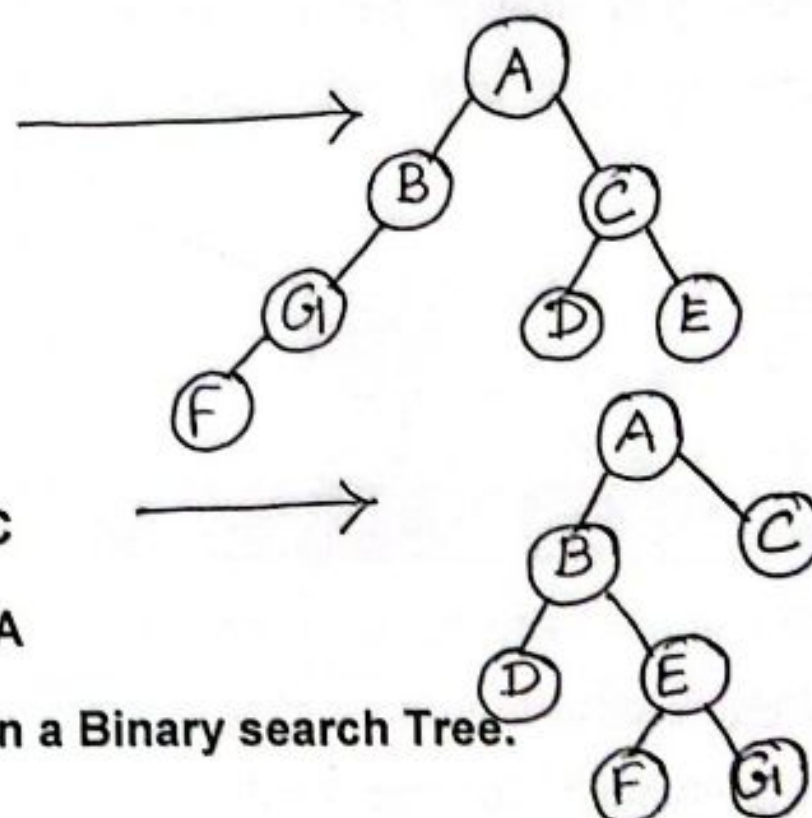
Inorder Traversal: F, G, B, A, D, C, E

Preorder Traversal: A, B, F, G, C, D, E

(C) Design the tree if:

Inorder traversal: D, B, F, E, G, A, C

Post order traversal: D, F, G, E, B, C, A



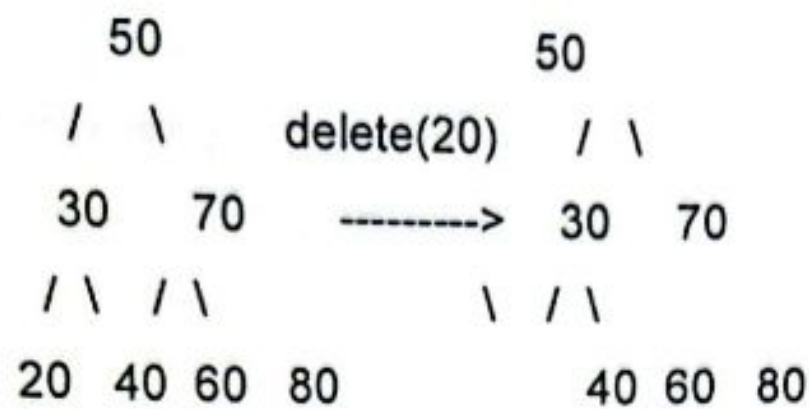
7. Write an Algorithm for deletion in a Binary search Tree.

Binary Search Tree is a node-based binary tree data structure which has the following properties:

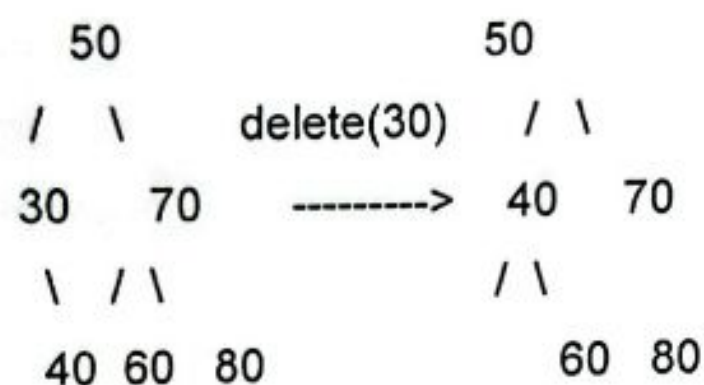
1. The left subtree of a node contains only nodes with keys lesser than the node's key.
2. The right subtree of a node contains only nodes with keys greater than the node's key.
3. The left and right subtree each must also be a binary search tree.

When we delete a node, three possibilities arise.

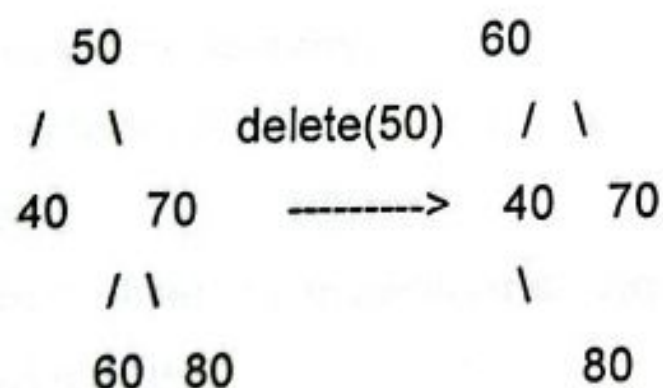
1) Node to be deleted is leaf: Simply remove from the tree.



2) Node to be deleted has only one child: Copy the child to the node and delete the child

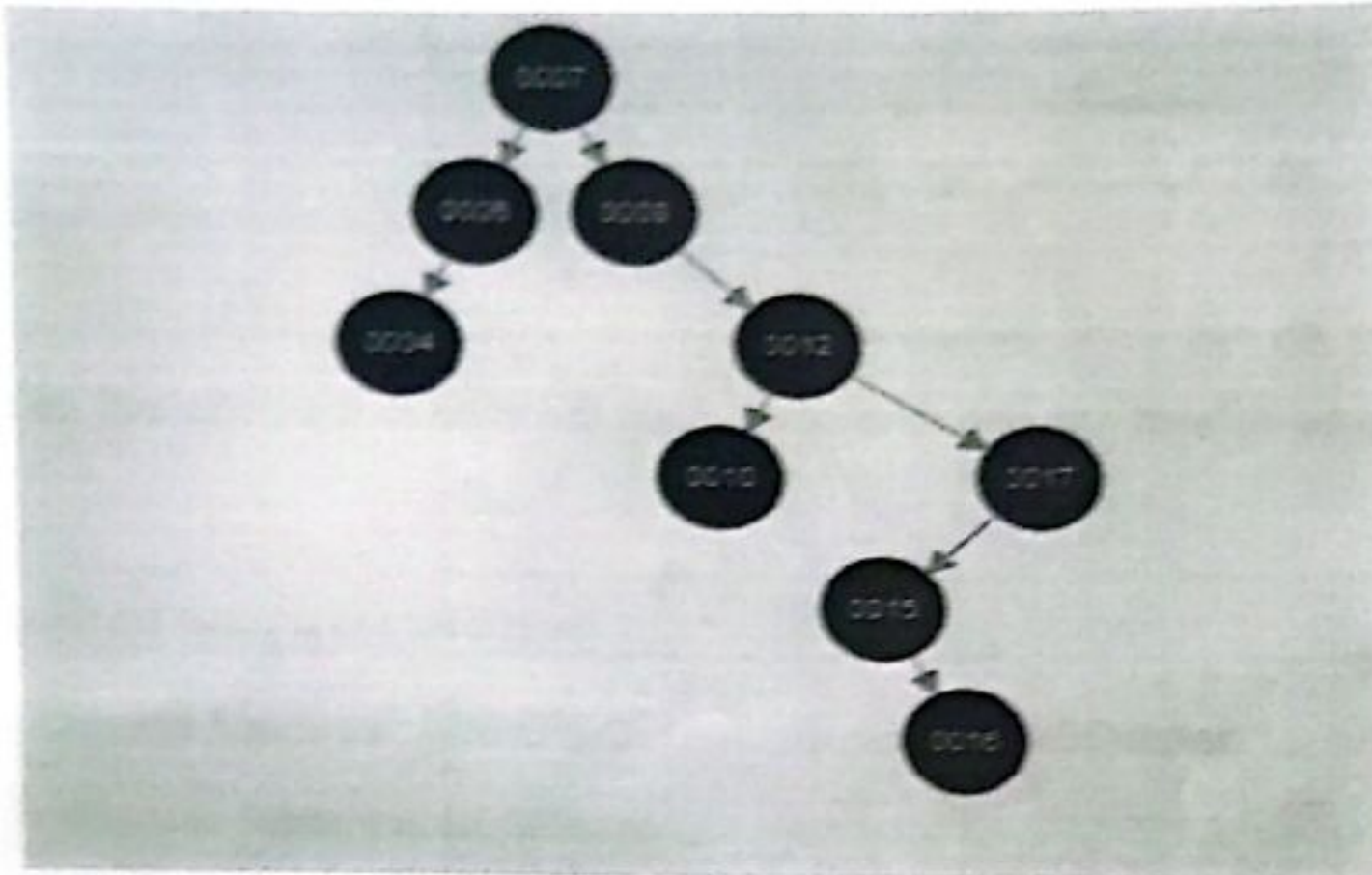


3) Node to be deleted has two children: Find inorder successor of the node. Copy contents of the inorder successor to the node and delete the inorder successor. Note that inorder predecessor can also be used.



8. Show the Binary search tree built from a sequence of insertions for the following sequence of keys(May-2018).

7,8,12,17,10,15,6,4,16.



9. What is binary search tree? Write a C program to perform the following operations in a Binary search Tree.(April-2017,Nov-2019)

(a) Insert

(b) Delete

(c) Find Min

(d) Find Max.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {
```

```
    int key;
```

```
    struct node *left, *right;
```

```
};
```



```

struct node* newNode(int item)
{
    struct node* temp
        = (struct node*)malloc(sizeof(struct node));
    temp->key = item;
    temp->left = temp->right = NULL;
    return temp;
}

void inorder(struct node* root)
{
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->key);
        inorder(root->right);
    }
}

struct node* insert(struct node* node, int key)
{
    if (node == NULL)
        return newNode(key);
    if (key < node->key)
        node->left = insert(node->left, key);
    else
        node->right = insert(node->right, key);
    return node;
}

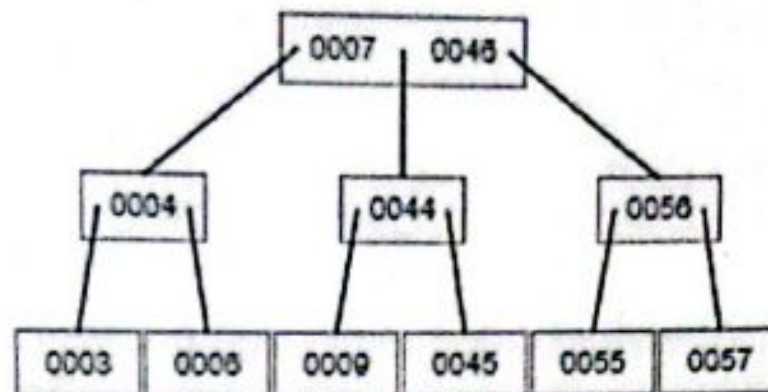
struct node* minValueNode(struct node* node)
{
    struct node* current = node;
    while (current && current->left != NULL)
        current = current->left;
}

```



```
return current;
}
struct node* deleteNode(struct node* root, int key)
{
    // base case
    if (root == NULL)
        return root;
    if (key < root->key)
        root->left = deleteNode(root->left, key);
    else if (key > root->key)
        root->right = deleteNode(root->right, key);
    else {
        // node with only one child or no child
        if (root->left == NULL) {
            struct node* temp = root->right;
            free(root);
            return temp;
        }
        else if (root->right == NULL) {
            struct node* temp = root->left;
            free(root);
            return temp;
        }
        struct node* temp = minValueNode(root->right);
        root->key = temp->key;
        root->right = deleteNode(root->right, temp->key);
    }
    return root;
}
```


10. Construct B tree to insert the following elements (order of B tree is 3) 55, 44, 3, 6, 7, 9, 45, 46, 56, 57 (April-2017)



11. Define AVL tree. Obtain the AVL tree inserting one integer at a time in the following sequence: 150, 155, 160, 115, 110, 140, 120, 145, 130, 140, 170, 180 (Nov 2019).

AVL tree is a binary search tree in which the difference of heights of left and right subtrees of any node is less than or equal to one. The technique of balancing the height of binary trees was developed by Adelson, Velskii, and Landi and hence given the short form as AVL tree or Balanced Binary Tree.

An AVL tree can be defined as follows:

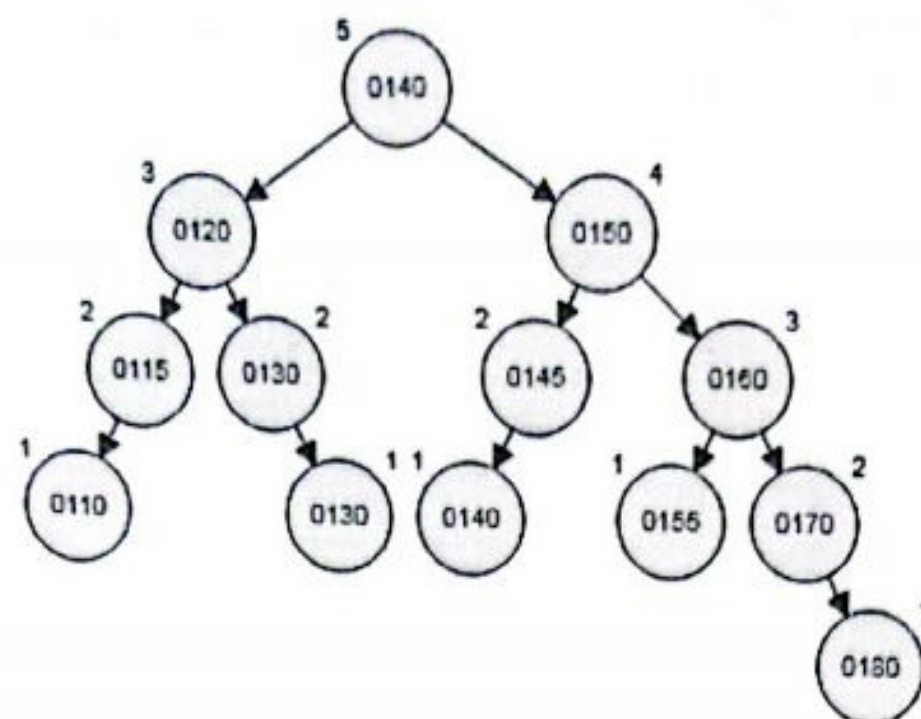
Let T be a non-empty binary tree with TL and TR as its left and right subtrees.

The tree is height balanced if:

1. TL and TR are height balanced
2. $h_L - h_R \leq 1$, where h_L - h_R are the heights of TL and TR

The Balance factor of a node in a binary tree can have value 1, -1, 0, depending on whether the height of its left subtree is greater, less than or equal to the height of the right subtree.

Since AVL trees are height balance trees, operations like insertion and deletion have low time complexity. Let us build an AVL tree: 150, 155, 160, 115, 110, 140, 120, 145, 130, 140, 170, 180



If you have the following tree having keys 1, 2, 3, 4, 5, 6, 7 and then the binary tree will be like the second figure:

12. For the arithmetic expression $((A*(B-C))(D-E)*(F+G-H))$. Construct the binary tree and apply the three tree traversal algorithm and obtain the result of traversal. (Nov 2017).

Before having the binary tree let us convert the given expression into postfix expression so that it will be easy for us to construct the expression tree,

The Postfix expression is : $ABC-^*DE-FG+H-^*$

13. Explain in detail the concept of B tree and construct a B tree of order 3 with the following set of data and insert them in the order as they appear: 30, 60, 50, 40, 10, 70, 20, 80, 90 (Nov-2017)

B-Tree is a self-balancing search tree. In most of the other self-balancing search trees (like AVL and Red-Black Trees), it is assumed that everything is in main memory. To understand the use of B-Trees, we must think of the huge amount of data that cannot fit in main memory. When the number of keys is high, the data is read from disk in the form of blocks. Disk access time is very high compared to the main memory access time. The main idea of using B-Trees is to reduce the number of disk accesses. Most of the tree operations (search, insert, delete, max, min, etc.,) require $O(h)$ disk accesses where h is the height of the tree. B-tree is a fat tree. The height of B-Trees is kept low by putting maximum

possible keys in a B-Tree node. Generally, the B-Tree node size is kept equal to the disk block size. Since the height of the B-tree is low so total disk accesses for most of the operations are reduced significantly compared to balanced Binary Search Trees like AVL Tree, Red-Black Tree, etc.

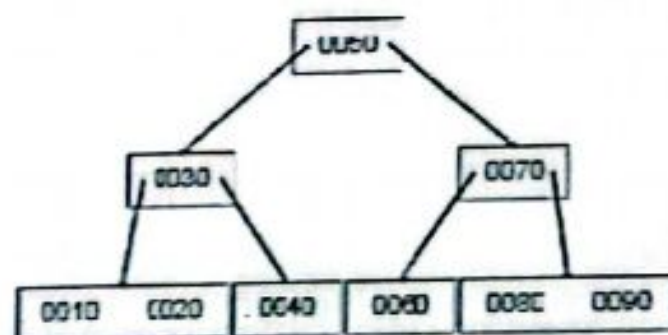
Properties of B-Tree:

1. All leaves are at the same level.
2. A B-Tree is defined by the term minimum degree 't'. The value of t depends upon disk block size.
3. Every node except root must contain at least $\lceil (t-1)/2 \rceil$ keys. The root may contain minimum 1 key.
4. All nodes (including root) may contain at most $t - 1$ keys..
5. Number of children of a node is equal to the number of keys in it plus 1.
6. All keys of a node are sorted in increasing order. The child between two keys k_1 and k_2 contains all keys in the range from k_1 and k_2 .

B-Tree grows and shrinks from the root which is unlike Binary Search Tree.

Binary Search Trees grow downward and also shrink from downward.

Like other balanced Binary Search Trees, time complexity to search, insert and delete is $O(\log n)$.



14. Give notes on (a) Tree Traversal (b) Binary search Tree (Nov-2015).

Unlike linear data structures (Array, Linked List, Queues, Stacks, etc) which have only one logical way to traverse them, trees can be traversed in different ways. Following are the generally used ways for traversing trees.

Algorithm Inorder(tree)

1. Traverse the left subtree, i.e., call Inorder(left-subtree)
 2. Visit the root.
 3. Traverse the right subtree, i.e., call Inorder(right-subtree)
- (Practice):

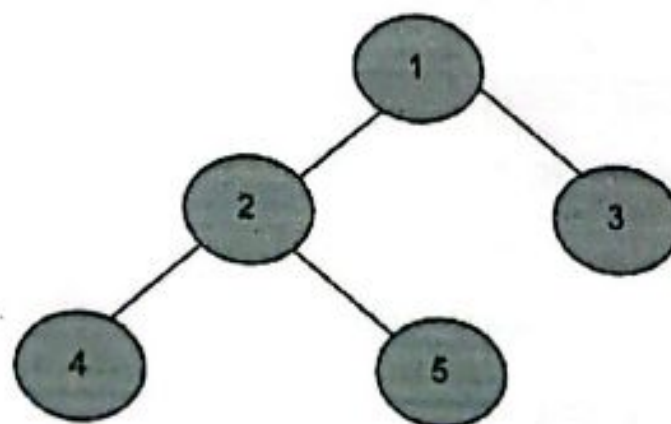
Algorithm Preorder(tree)

1. Visit the root.
2. Traverse the left subtree, i.e., call Preorder(left-subtree)
3. Traverse the right subtree, i.e., call Preorder(right-subtree)

Postorder Traversal (Practice):

Algorithm Postorder(tree)

- 1.. Traverse the left subtree, i.e., call Preorder(left-subtree)
3. Traverse the right subtree, i.e., call Preorder(right-subtree)
3. Visit the root.



- (a) Inorder (Left, Root, Right): 4 2 5 1 3
(b) Preorder (Root, Left, Right): 1 2 4 5 3
(c) Postorder (Left, Right, Root): 4 5 2 3 1

Binary Search Tree is a node-based binary tree data structure which has the following properties:

- The left subtree of a node contains only nodes with keys lesser than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- The left and right subtree each must also be a binary search tree.

15. Explain the B + tree indexing and Trie tree indexing.(Nov 2015).

In order, to implement dynamic multilevel indexing, B-tree and B+ tree are generally employed. The drawback of B-tree used for indexing, however is that it stores the data pointer (a pointer to the disk file block containing the key value), corresponding to a particular key value, along with that key value in the node of a B-tree. This technique, greatly reduces the number of entries that can be packed into a node of a B-tree, thereby contributing to the increase in the number of levels in the B-tree, hence increasing the search time of a record.

B+ tree eliminates the above drawback by storing data pointers only at the leaf nodes of the tree. Thus, the structure of leaf nodes of a B+ tree is quite different from the structure of internal nodes of the B tree. It may be noted here that, since data pointers are present only at the leaf nodes, the leaf nodes must necessarily store all the key values along with their corresponding data pointers to the disk file block, in order to access them. Moreover, the leaf nodes are linked to provide ordered access to the records. The leaf nodes, therefore form the first level of index, with the internal nodes forming the other levels of a multilevel index. Some of the key values of the leaf nodes also appear in the internal nodes, to simply act as a medium to control the searching of a record.